

Министерство науки и высшего образования
Российской Федерации

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«Российский государственный педагогический
университет им. А. И. Герцена»

Институт информационных технологий и
технологического образования
Кафедра информационных технологий и электронного
обучения

Доклад

по дисциплине «Информационные технологии»

Тема: Функциональное программирование

Выполнил:

обучающийся 1 курса
Сурков Андрей Анатольевич

Руководитель:

ассистент кафедры ИТиЭО
Иванова Е.А.

Санкт-Петербург
2023

Оглавление

0.1	Введение	2
0.2	Термины и определения	3
0.3	Функциональное программирование	5
0.4	История функционального программирования	6
0.5	Отличительные черты функциональных языков программирования	8
0.6	Преимущества функционального программирования	9
0.7	Недостатки функционального программирования	10
0.8	Заключение	11
0.9	Таблица сравнения парадигм	12
0.10	Пример архитектуры функциональной программы	13
0.11	Литература	14

0.1. Введение

Функциональное программирование является одной из парадигм программирования и противопоставляется императивному программированию. Для эффективного решения различных задач, связанных с написанием кода, необходимо чётко понимать, какой парадигме следует придерживаться в каждом конкретном случае, и на основе этого решения выбирать язык программирования для написания кода, а также стиль и методы работы над программой. Для осознанного принятия такого решения необходимо ознакомиться с основными концепциями, принципами и историей функционального программирования.

Актуальность данной работы заключается в необходимости повышения осведомлённости о функциональном программировании у программистов для более эффективного написания программ, откуда вытекает и цель работы: в кратком и понятном формате представить историю возникновения и развития функционального программирования, рассказать об основных концепциях данной парадигмы, её достоинствах и недостатках.

0.2. Термины и определения

- **ИИ (Искусственный интеллект)** — интеллект, демонстрируемый машинами, то есть способность машин собирать информацию и делать из неё выводы.
- **Константа** — некоторая величина, не изменяющая свое значение в рамках рассматриваемого процесса.
- **Мемоизация** — сохранение результатов выполнения функций для предотвращения повторных вычислений.
- **Парадигма** — это совокупность идей и понятий, определяющих стиль написания компьютерных программ (подход к программированию).
- **Переменная** — область памяти, адрес которой можно использовать для осуществления доступа к данным и изменять значение в ходе выполнения программы.
- **Программа** — комбинация компьютерных инструкций и данных, позволяющая аппаратному обеспечению вычислительной системы выполнять вычисления или функции управления.
- **Рекурсия** — вызов функции (процедуры) из неё же самой, непосредственно (простая рекурсия) или через другие функции (сложная или косвенная рекурсия).
- **Функция** — фрагмент программного кода, к которому можно обратиться из другого места программы.
- **Цикл** — многократно исполняемая последовательность инструкций.

- **ЭВМ (Электронная вычислительная машина)** — комплекс технических, аппаратных и программных средств, предназначенных для автоматической обработки информации, вычислений, автоматического управления.

0.3. Функциональное программирование

Функциональное программирование — одна из первых парадигм в программировании, основной концепцией которой является использование функций как базовых строительных блоков программы. Вычисление результата программы осуществляется путём вычисления результатов функций от исходных данных и результатов других функций, без сохранения промежуточных результатов в качестве переменных (как это происходит в императивном программировании). Вместо этого, функциональные языки программирования могут сохранять «таблицу» входных значений и результатов функций, что значительно ускоряет работу программы в случае работы с большими объёмами данных, которые могут повторяться или пересекаться. Такая система возможна благодаря тому, что в функциональных языках программирования результат работы функций зависит только от входных констант. У программы нет переменных и «состояний», что делает её работу более надёжной и предсказуемой.

0.4. История функционального программирования

Основой парадигмы функционального программирования стали математические абстракции «Лямбда-исчисление» и «Комбинаторная логика», которые описывали сложные явления через фундаментальные математические понятия и теории, описывали их «языком» математической логики. Комбинаторная логика была введена М. Шейнфинкелем в 1920 году, а концепция лямбда-исчисления была представлена Алонзо Чёрчем в 1936 году в статье «An Unsolvability Problem of Elementary Number Theory». И λ -исчисление, и комбинаторная логика были разработаны для более ясного и точного описания принципов и основ математики.

Первым функциональный язык программирования был разработан в 1958 году американским программистом Джоном Маккарти и назывался «Лисп». Название языка расшифровывается и переводится как «язык обработки списков», а символ содержит два символа лямбда. Лисп был создан для разработки искусственного интеллекта и является высокоуровневым языком программирования, который используется и в наше время. Широкие возможности для автоматического распараллеливания вычислений у функциональных языков программирования сделали эту концепцию предпочтительной для разработки ИИ.

Последующие функциональные языки программирования были узконаправленными и использовались для разработки программных решений задач в конкретной сфере. Например, язык APL, созданный в 1964 году программистом Кеннетом Айверсоном, широко использовался в различных системах компьютерной алгебры, а язык программирования «К» часто использовался для написания программ, помогающих сотрудникам финансового сектора.

Наиболее известный в наше время функциональный язык про-

граммирования «Haskell» был разработан командой опытных разработчиков в 1990 году и основывался на другом языке программирования — «Miranda», выпущенном в 1985 году. Haskell отличается полным исключением императивности, а также строгой типизацией.

0.5. Отличительные черты функциональных языков программирования

Одной из основ функциональных языков программирования являются «функции высших порядков». Функции высших порядков — это такие функции, которые могут принимать в качестве аргументов и возвращать другие функции.

Другим немаловажным аспектом, который присущ функциональным языкам программирования, являются «чистые функции». Чистые функции — это такие функции в программе, на результат работы которых влияют только входные константы, то есть их работа не зависит от части остальной программы. Чистые функции обладают некоторыми полезными свойствами, такими как: возможность мемоизации, возможность поменять порядок выполнения функций или распараллелить их, большая свобода компилятора для комбинирования и реорганизации вычисления выражений в программе.

В функциональном программировании цикл обычно реализуется в виде хвостовой рекурсии, что также является одной из особенностей этой парадигмы. Хвостовая рекурсия может быть распознана и оптимизирована компилятором в код, получаемый после компиляции аналогичной итерации в императивном языке программирования.

0.6. Преимущества функционального программирования

В силу отсутствия у программы состояний программы в функциональном программировании отличаются надёжностью кода, а также лёгкостью поиска и устранения ошибок.

Чистые функции предоставляют большой уровень свободы компилятору для реструктуризации программы, что позволяет описывать программу в так называемом «декларативном» виде, когда жёсткая последовательность выполнения многих операций, необходимых для вычисления результата, в явном виде не задаётся, а формируется автоматически в процессе вычисления функций.

Помимо изменения порядка выполнения функций, в функциональных языках программирования удобно распараллеливать вычисления, что возможно даже в автоматическом режиме.

0.7. Недостатки функционального программирования

Программы, написанные на функциональных языках программирования, вынуждены постоянно выделять и освобождать память для передачи результатов работы функций, что экономит оперативную память ЭВМ, однако значительно замедляет работу программы.

Непредсказуемый порядок вызова функций может приводить к проблемам на этапе ввода и вывода значений.

0.8. Заключение

Функциональные языки программирования являются интересной альтернативой императивным языкам и имеют свои преимущества и недостатки. Функциональные языки программирования обладают меньшей популярностью, чем императивные, однако они значительно упрощают решение некоторых задач. Несмотря на меньшую популярность, функциональные языки программирования продолжают развиваться и совершенствоваться, смягчая свои недостатки и подчёркивая сильные стороны, что делает их актуальными и незаменимыми и по сей день.

0.9. Таблица сравнения парадигм

Функциональное программирование	Императивное прог
Использует чистые функции, отсутствие состояний	Использует изменяемые со
Основное внимание на результатах функций	Основное внимание на по
Возможность распараллеливания вычислений	Более сложное расп
Легче отлаживать и тестировать программы	Сложнее отлаживать из-за и

Таблица 1: Сравнение функционального и императивного программирования

0.10. Пример архитектуры функциональной программы

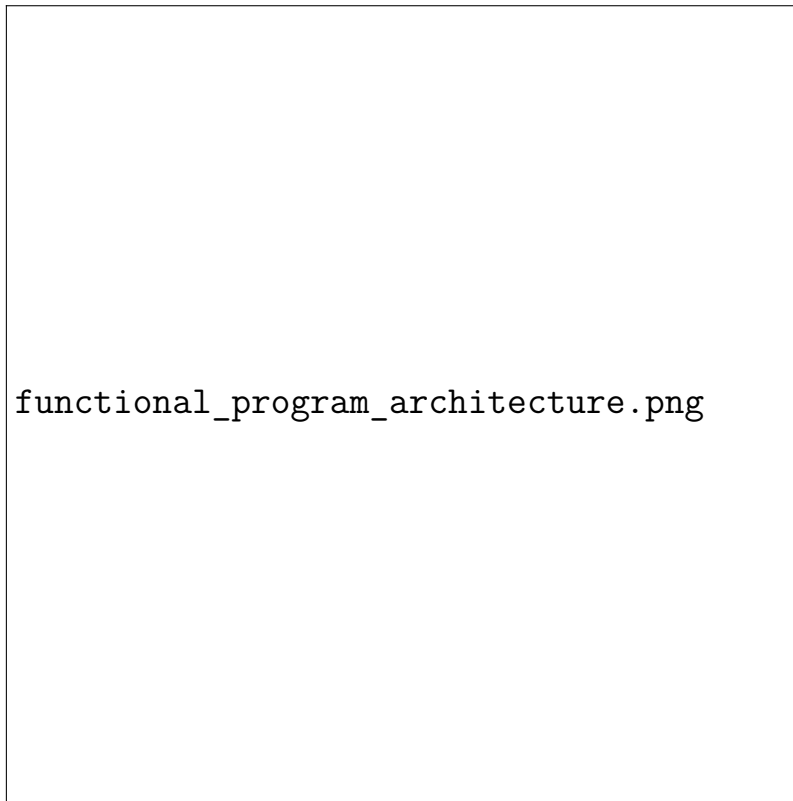


Рис. 1: Пример архитектуры функциональной программы

0.11. Литература

1. Church, Alonzo. An Unsolvble Problem of Elementary Number Theory // American Journal of Mathematics. — 1936. — Vol. 58. — P. 345—363.
2. Роджер Пенроуз. Лямбда-исчисление Черча // Новый ум короля: О компьютерах, мышлении и законах физики. — Едиториал УРСС, 2003.
3. McCarthy, John. History of Lisp // In Association for Computing Machinery SIGPLAN History of Programming Languages Conference. — 1978.
4. Потапенко, В. А. Техники функционального программирования. URL: <https://wm-help.net/download.php?id=49.Tailcalloptimization.UR>
[http : //wiki.c2.com/?TailCallOptimization](http://wiki.c2.com/?TailCallOptimization).
5. ISO/IEC/IEEE 24765:2010 Systems and software engineering — Vocabulary.